ML Problem Solving: Implementation of Perceptron, Adaline, MLP

Allowed Libraries: `numpy`, `matplotlib`, and `scikit-learn` (strictly for dataset generation, train/test splitting, and evaluation metrics). **PyTorch, TensorFlow, JAX, Autograd, and all other automated differentiation or deep learning frameworks are strictly prohibited.**

Overview

Module	Focus Topics
Module 1: Binary Perceptron	Hyperplane geometry, Rosenblatt update rule, margin-based updates, linearly separable limits.
Module 2: Multiclass Perceptron	Multi-prototype competitive boundaries, argmax winner-take-all dynamics, Kesler's reduction.
Module 3: ADALINE and Optimization	Widrow-Hoff Delta rule, quadratic loss surfaces, feature scaling, mini-batch gradient descent.
Module 4: Single-Layer Sigmoidal Unit	Sigmoid activation, logit geometry, binary cross-entropy (BCE) loss surface, vectorized gradient descent.
Module 5: Two-Layer MLP (Sigmoid + Softmax)	Matrix calculus, forward propagation, shift-invariant Softmax, backpropagation derivation, gradient verification.

Mathematical Notation and Architectural Foundations

This write-up adheres to the following linear algebra and calculus conventions:

- Vectors are treated as column vectors unless explicitly written as rows: $\mathbf{x} \in \mathbb{R}^{d \times 1}$.
- A design matrix representing N samples in a d-dimensional feature space is denoted:

$$X \in \mathbb{R}^{N \times d} = \begin{bmatrix} \mathbf{x}_1^T \\ \mathbf{x}_2^T \\ \vdots \\ \mathbf{x}_N^T \end{bmatrix}$$

- Model parameters are separated explicitly into weight matrices (or vectors) and bias vectors. We do not use the augmented bias trick ($x_0 = 1$) in order to ensure clean separation between linear transformations and translation terms.
- Element-wise operations (Hadamard product) are denoted by $\odot$.
- All vector or matrix derivatives follow denominator layout unless otherwise specified. For scalar loss $\mathcal{L}$ and matrix $W \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$, the gradient matrix $\frac{\partial \mathcal{L}}{\partial W}$ maintains dimensions identical to W:

$$\left[\frac{\partial \mathcal{L}}{\partial W}\right]_{ij} = \frac{\partial \mathcal{L}}{\partial W_{ij}}$$

1. Module 1: The Binary Perceptron from First Principles

1.1 Geometric Derivation and the Rosenblatt Rule

The classical Rosenblatt Perceptron constructs an affine hyper-surface in $\mathbb{R}^d$:

$$z = \mathbf{w}^T\mathbf{x} + b = \sum_{j=1}^{d} w_j x_j + b$$

The decision surface is the hyperplane $\mathcal{H} = \{\mathbf{x} \in \mathbb{R}^d \mid \mathbf{w}^T\mathbf{x} + b = 0\}$. The normal vector $\mathbf{w}$ points toward the positive half-space. The activation function is the non-smooth Heaviside step:

$$\phi(z) = \begin{cases} +1 & \text{if } z \geq 0 \\ -1 & \text{if } z < 0 \end{cases}$$

Given a training set $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ with labels $y_i \in \{-1, +1\}$, a classification error occurs if and only if:

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) \leq 0$$

This is different than the previous convention used in the lecture notes and problem-solving, where $y_i \in \{0, 1\}$.

When an error occurs on sample $(\mathbf{x}_i, y_i)$, the Rosenblatt update rule adjusts the parameters using a learning rate $\eta \in (0, 1]$:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + \eta\, y_i\, \mathbf{x}_i$$

$$b^{(t+1)} = b^{(t)} + \eta\, y_i$$

1.2 Task Specification

Implement the binary perceptron network from scratch using standard NumPy operations according to the following specifications:

1. Construct the model using explicit separating parameters: a weight vector $\mathbf{w} \in \mathbb{R}^d$ initialized to zero (or small random floats) and a scalar bias $b \in \mathbb{R}$ initialized to 0.0.
2. Implement the forward pass to compute linear scores $z_i = \mathbf{x}_i^T\mathbf{w} + b$ and apply the step activation function yielding binary predictions $\hat{y}_i \in \{-1, +1\}$.
3. Implement sample-by-sample training using the Rosenblatt update rule triggered strictly upon classification errors $y_i z_i \leq 0$.
4. Track the number of misclassifications per epoch and provide early termination when zero errors are achieved.
5. Evaluate the model on a linearly separable 2D dataset generated via `sklearn.datasets.make_blobs` and confirm convergence within finite epochs.

2. Module 2: The Multiclass Perceptron (Kesler's Construction)

2.1 Multi-Prototype Geometry

For a C-class classification problem, $y_i \in \{0, 1, \ldots, C-1\}$, we assign an independent weight vector $\mathbf{w}_c \in \mathbb{R}^d$ and bias $b_c \in \mathbb{R}$ to each class $c \in \{0, \ldots, C-1\}$. We organize these parameters into a weight matrix $W \in \mathbb{R}^{d \times C}$ and bias vector $\mathbf{b} \in \mathbb{R}^C$:

$$\mathbf{z} = \mathbf{x}^T W + \mathbf{b}^T \in \mathbb{R}^{1 \times C}$$

The predicted class $\hat{y}$ is selected via the winner-take-all decision rule:

$$\hat{y} = \arg \max_{c \in \{0,\ldots,C-1\}} z_c = \arg\max_c \left(\mathbf{w}_c^T \mathbf{x} + b_c\right)$$

The decision regions $\mathcal{R}_c = \{\mathbf{x} \in \mathbb{R}^d \mid \mathbf{w}_c^T \mathbf{x} + b_c > \mathbf{w}_k^T \mathbf{x} + b_k,\ \forall k \neq c\}$ form convex polyhedra bounded by piecewise linear hyperplanes.

2.2 Generalized Winner-Take-All Update Rule

When a sample $(\mathbf{x}_i, y_i)$ is processed, if the predicted label $\hat{y} = \arg\max_c z_c$ does not match true label y_i, parameters are updated by promoting the true class prototype and penalizing the predicted competing prototype:

$$\mathbf{w}_{y_i}^{(t+1)} = \mathbf{w}_{y_i}^{(t)} + \eta\, \mathbf{x}_i, \quad b_{y_i}^{(t+1)} = b_{y_i}^{(t)} + \eta$$

$$\mathbf{w}_{\hat{y}}^{(t+1)} = \mathbf{w}_{\hat{y}}^{(t)} - \eta\, \mathbf{x}_i, \quad b_{\hat{y}}^{(t+1)} = b_{\hat{y}}^{(t)} - \eta$$

All other class weights $\mathbf{w}_k$ for $k \notin \{y_i, \hat{y}\}$ remain unchanged.

2.3 Task Specification

Implement the multiclass perceptron network using a multi-prototype architecture according to the following specifications:

1. Construct the weight matrix $W \in \mathbb{R}^{d \times C}$ and bias vector $\mathbf{b} \in \mathbb{R}^C$, initializing both to zeros.
2. Formulate vectorized linear score evaluations across classes: $Z = XW + \mathbf{b}^T$.
3. Compute predictions via $\arg\max$ along the class axis.
4. Execute the competitive dual-update rule: upon an error where $\hat{y} \neq y$, add $\eta \mathbf{x}_i$ to the true class parameters and subtract $\eta \mathbf{x}_i$ from the false winning class parameters.
5. Train the architecture on a 3-class synthetic classification dataset generated with `sklearn.datasets.make_blobs` and verify plotting the multiclass decision boundaries.

3. Module 3: ADALINE and Optimization via Gradient Descent

3.1 Widrow-Hoff Delta Rule and the Continuous Objective

The Adaptive Linear Neuron (ADALINE) separates the linear continuous output from the discrete step activation. It defines an explicit continuous loss function over continuous linear predictions:

$$\hat{y}_{\text{linear}} = \mathbf{w}^T \mathbf{x} + b$$

The objective is the Mean Squared Error (MSE) loss:

$$\mathcal{L}_{\text{MSE}}(\mathbf{w}, b) = \frac{1}{2N} \sum_{i=1}^{N} \left(y_i - (\mathbf{w}^T \mathbf{x}_i + b) \right)^2$$

Unlike Rosenblatt's Perceptron, ADALINE's loss surface is quadratic and globally convex. Computing the analytical gradients with respect to $\mathbf{w}$ and b:

$$\nabla_{\mathbf{w}} \mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \left(y_i - (\mathbf{w}^T \mathbf{x}_i + b) \right) \mathbf{x}_i = -\frac{1}{N} X^T (\mathbf{y} - \hat{\mathbf{y}}_{\text{linear}})$$

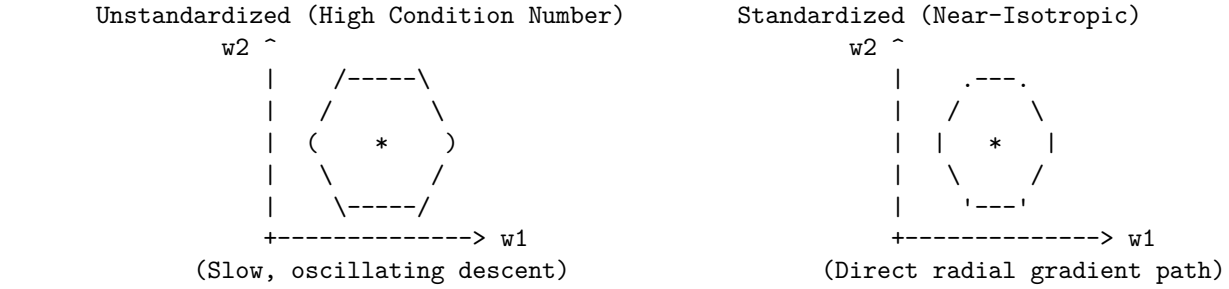
$$\frac{\partial \mathcal{L}}{\partial b} = -\frac{1}{N} \sum_{i=1}^{N} \left(y_i - (\mathbf{w}^T \mathbf{x}_i + b) \right) = -\frac{1}{N} \mathbf{1}^T (\mathbf{y} - \hat{\mathbf{y}}_{\text{linear}})$$

3.2 Impact of Feature Scaling on the Condition Number

Let the Hessian matrix of the MSE loss be:

$$H = \nabla^2 \mathcal{L} = \frac{1}{N} X^T X$$

The condition number $\kappa(H) = \frac{\lambda_{\max}(H)}{\lambda_{\min}(H)}$ determines the curvature eccentricity of the loss contours. If features have unequal variance, $\kappa(H) \gg 1$, yielding elongated elliptical contours that cause standard gradient descent to oscillate perpendicularly to the optimal descent path. Standardization ensures $\kappa(H) \approx 1$, producing spherical contours that allow fast, direct convergence.



3.3 Task Specification

Implement the ADALINE network using mini-batch gradient descent according to the following specifications:

1. Implement standard feature scaling (z-score standardization) to center input features at zero mean and unit variance.
2. Implement linear activation for the continuous forward pass: $\hat{\mathbf{y}}_{\text{linear}} = X\mathbf{w} + b$.
3. Compute the Mean Squared Error (MSE) loss function value at each epoch to monitor convergence.
4. Implement mini-batch gradient descent: partition the dataset into randomized mini-batches of size B, calculate exact analytical matrix gradients over each mini-batch, and update parameters:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}, \quad b \leftarrow b - \eta \frac{\partial \mathcal{L}}{\partial b}$$

5. Using the same dataset as in the previous module, show per-mini-batch MSE and accuracy until convergence. Use $z \geq 0$ to perform binary classifications $y \in \{-1, +1\}$.

4. Module 4: Single-Layer Perceptron with Sigmoid Activation and BCE Loss

4.1 Sigmoid Activation and Probabilistic Logits

To output calibrated posterior probabilities $P(y = 1 \mid \mathbf{x})$, we pass the linear net input $z = \mathbf{w}^T\mathbf{x} + b$ through the nonlinear Sigmoid activation function $\sigma(z)$:

$$\hat{y} = a = \sigma(z) = \frac{1}{1 + e^{-z}}$$

The derivative of the Sigmoid function satisfies the identity:

$$\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z)) = a(1 - a)$$

4.2 Binary Cross-Entropy Loss and Gradient Derivation

Assuming Bernoulli targets $y_i \in \{0, 1\}$, the likelihood of observing label y_i given prediction $a_i = \sigma(z_i)$ is:

$$P(y_i \mid \mathbf{x}_i) = a_i^{y_i}(1 - a_i)^{1-y_i}$$

Taking the negative log-likelihood over N samples produces the Binary Cross-Entropy (BCE) loss:

$$\mathcal{L}_{\text{BCE}}(\mathbf{w}, b) = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \ln(a_i) + (1 - y_i)\ln(1 - a_i)\right]$$

Applying the chain rule to compute $\frac{\partial \mathcal{L}}{\partial \mathbf{w}}$:

$$\frac{\partial \mathcal{L}}{\partial a_i} = -\frac{1}{N}\left[\frac{y_i}{a_i} - \frac{1 - y_i}{1 - a_i}\right] = -\frac{1}{N}\frac{y_i - a_i}{a_i(1 - a_i)}$$

$$\frac{\partial a_i}{\partial z_i} = a_i(1 - a_i)$$

$$\frac{\partial z_i}{\partial \mathbf{w}} = \mathbf{x}_i$$

Multiplying the terms:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \sum_{i=1}^{N}\frac{\partial \mathcal{L}}{\partial a_i}\frac{\partial a_i}{\partial z_i}\frac{\partial z_i}{\partial \mathbf{w}} = \frac{1}{N}\sum_{i=1}^{N}(a_i - y_i)\mathbf{x}_i = \frac{1}{N}X^T(\mathbf{a} - \mathbf{y})$$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{1}{N}\sum_{i=1}^{N}(a_i - y_i) = \frac{1}{N}\mathbf{1}^T(\mathbf{a} - \mathbf{y})$$

The product $\frac{\partial \mathcal{L}}{\partial a_i}\frac{\partial a_i}{\partial z_i} = a_i - y_i$ cancels the denominator $a_i(1 - a_i)$, preventing saturation-induced vanishing gradients at the output unit.

4.3 Task Specification

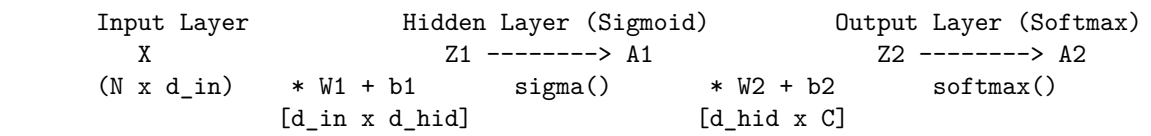
Implement a single-layer perceptron network with Sigmoid activation using the Binary Cross-Entropy (BCE) loss for gradient descent optimization according to the following specifications:

1. Implement a numerically safe Sigmoid function using clipping or piecewise formulation to eliminate floating-point overflow encountered when evaluating e^{-z} for large negative z.
2. Implement forward propagation yielding predicted probabilities $\mathbf{a} = \sigma(X\mathbf{w} + b)$.
3. Implement the BCE loss computation with numerical safeguards ($\epsilon = 10^{-15}$) to avoid evaluating $\ln(0)$.
4. Implement vectorized gradient descent updates using $\nabla_{\mathbf{w}}\mathcal{L} = \frac{1}{N}X^T(\mathbf{a} - \mathbf{y})$ and $\frac{\partial \mathcal{L}}{\partial b} = \frac{1}{N}\sum_i(a_i - y_i)$.
5. Train the network on the same binary classification dataset as in previous modules and evaluate classification accuracy by thresholding probabilities at $a \geq 0.5$.

5. Module 5: Two-Layer Perceptron (Sigmoid Hidden, Softmax Output)

5.1 Architecture and Matrix Equations

We construct a fully connected two-layer neural network with input dimension d_{in}, hidden layer dimension d_{hid}, and output dimension C:



- **Hidden Layer:**

$$Z_1 = XW_1 + \mathbf{1}_N \mathbf{b}_1^T \in \mathbb{R}^{N \times d_{\text{hid}}}$$

$$A_1 = \sigma(Z_1) = \frac{1}{1 + \exp(-Z_1)} \in \mathbb{R}^{N \times d_{\text{hid}}}$$

- **Output Layer:**

$$Z_2 = A_1 W_2 + \mathbf{1}_N \mathbf{b}_2^T \in \mathbb{R}^{N \times C}$$

$$A_2 = \text{Softmax}(Z_2) \in \mathbb{R}^{N \times C}$$

To prevent numerical overflow in the exponential calculation, Softmax is computed using the shift-invariance identity $\text{Softmax}(\mathbf{z}) = \text{Softmax}(\mathbf{z} - \max(\mathbf{z}))$:

$$[A_2]_{ic} = \frac{\exp\left([Z_2]_{ic} - \max_k [Z_2]_{ik}\right)}{\sum_{j=1}^{C} \exp\left([Z_2]_{ij} - \max_k [Z_2]_{ik}\right)}$$

5.2 Categorical Cross-Entropy Loss

Given one-hot encoded ground truth labels $Y \in \{0, 1\}^{N \times C}$, the average Categorical Cross-Entropy loss is:

$$\mathcal{L}_{\text{CCE}}(W_1, \mathbf{b}_1, W_2, \mathbf{b}_2) = -\frac{1}{N} \sum_{i=1}^{N} \sum_{c=1}^{C} Y_{ic} \ln([A_2]_{ic})$$

5.3 Complete Backpropagation Derivation

We apply matrix calculus using denominator layout:

1. **Output Error Tensor (δ_2):**

$$\delta_2 = \frac{\partial \mathcal{L}}{\partial Z_2} = \frac{1}{N}(A_2 - Y) \in \mathbb{R}^{N \times C}$$

2. **Output Layer Parameter Gradients:**

$$\frac{\partial \mathcal{L}}{\partial W_2} = A_1^T \delta_2 \in \mathbb{R}^{d_{\text{hid}} \times C}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}_2} = \delta_2^T \mathbf{1}_N = \sum_{i=1}^{N} (\delta_2)_{i,:} \in \mathbb{R}^{C \times 1}$$

3. **Hidden Layer Error Propagation (δ_1):** Using the multivariate chain rule:

$$\frac{\partial \mathcal{L}}{\partial A_1} = \delta_2 W_2^T \in \mathbb{R}^{N \times d_{\text{hid}}}$$

$$\delta_1 = \frac{\partial \mathcal{L}}{\partial Z_1} = \frac{\partial \mathcal{L}}{\partial A_1} \odot \sigma'(Z_1) = \left(\delta_2 W_2^T\right) \odot [A_1 \odot (\mathbf{1} - A_1)] \in \mathbb{R}^{N \times d_{\text{hid}}}$$

4. **Hidden Layer Parameter Gradients:**

$$\frac{\partial \mathcal{L}}{\partial W_1} = X^T \delta_1 \in \mathbb{R}^{d_{\text{in}} \times d_{\text{hid}}}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}_1} = \delta_1^T \mathbf{1}_N = \sum_{i=1}^{N} (\delta_1)_{i,:} \in \mathbb{R}^{d_{\text{hid}} \times 1}$$

5.4 Parameter Initialization: Xavier (Glorot) Uniform

Initializing weights to zero causes symmetric hidden representations where all neurons compute identical updates. To preserve activation and gradient variance across layers, weights are initialized using Xavier uniform bounds:

$$W \sim \mathcal{U}\left(-\sqrt{\frac{6}{d_{\text{in}} + d_{\text{out}}}},\ +\sqrt{\frac{6}{d_{\text{in}} + d_{\text{out}}}}\right)$$

Biases are initialized to zeros: $\mathbf{b} = \mathbf{0}$.

5.5 Task Specification

Implement a complete two-layer neural network with Sigmoid hidden units and Softmax output units using pure NumPy according to the following specifications:

1. Initialize parameters $W_1, \mathbf{b}_1, W_2, \mathbf{b}_2$ using Xavier uniform distribution for weights and zeros for biases.
2. Implement forward propagation: compute hidden layer activations via the Sigmoid function and output probabilities via the numerically stable shift-invariant Softmax function.
3. Compute the Categorical Cross-Entropy (CCE) loss across training iterations.
4. Implement the full backward propagation pass: compute analytical error tensors δ_2 and δ_1, followed by parameter gradients $\nabla W_2, \nabla \mathbf{b}_2, \nabla W_1, \nabla \mathbf{b}_1$.
5. Update all parameters via gradient descent and train the architecture on the following non-linearly-separable 4-class dataset:

```
from sklearn.datasets import make_blobs
X, y = make_blobs(
    n_samples=600,
    centers=4,       # Four classes
    random_state=42, # Understanders will understand
    cluster_std=4.0  # High standard deviation forces the classes to overlap
)
```